Занятие 47.4. Указатели на указатели

Указатель на указатель — это именно то, что вы подумали: **указатель**, который содержит адрес другого указателя.

Оглавление:

1. Указатели на указатели
2. Массивы указателей
3. Двумерные динамически выделенные массивы
4. Указатель на указатель на указатель на указатель и т.д.
5. Заключение

Указатели на указатели

Обычный указатель типа int объявляется с использованием одной звёздочки:

```
int *ptr; // указатель типа int, одна звёздочка
```

Указатель на указатель типа int объявляется с использованием двух звёздочек:

```
int **ptrptr; // указатель на указатель типа int (две звёздочки)
```

Указатель на указатель работает подобно обычному указателю: вы можете его разыменовать для получения значения, на которое он указывает. И, поскольку этим значением является другой указатель, для получения исходного значения вам потребуется выполнить разыменование еще раз. Их следует выполнять последовательно:

```
#include <iostream>

int main()
{
    int value = 7;

    int *ptr = &value;
    std::cout << *ptr << std::endl; // разыменовываем указатель, чтобы получить значение типа int

    int **ptrptr = &ptr;
    std::cout << **ptrptr << std::endl;

    return 0;
}
```

Результат выполнения программы:

```
7
7
```

Обратите внимание, вы не можете инициализировать указатель на указатель напрямую значением:

```
int value = 7;
int **ptrptr = &&value; // нельзя
```

Это связано с тем, что оператор адреса (`&`) требует **l-value**, но `&value` — это r-value. Однако указателю на указатель можно задать **значение null**:

```
int **ptrptr = nullptr; // используйте 0, если не поддерживается C++11
```

Массивы указателей

Указатели на указатели имеют несколько применений. Наиболее используемым является **динамическое выделение** массива указателей:

```
int **array = new int*[20]; // выделяем массив из 20 указателей типа int
```

Это тот же обычный **динамически выделенный массив**, за исключением того, что элементами являются указатели на тип int, а не значения типа int.

Двумерные динамически выделенные массивы

Другим распространенным применением указателей на указатели является динамическое выделение **многомерных массивов**. В отличие от двумерного фиксированного массива, который можно легко объявить следующим образом:

```
int array[15][7];
```

Динамическое выделение двумерного массива немного отличается. У вас может возникнуть соблазн написать что-то вроде следующего:

```
int **array = new int[15][7]; // не будет работать!
```

Здесь вы получите ошибку. Есть два возможных решения. Если правый индекс является **константой типа compile-time**, то вы можете сделать следующее:

```
int (*array)[7] = new int[15][7];
```

Скобки здесь потребуются для соблюдения **приоритета**. В C++11 хорошей идеей будет использовать **ключевое слово auto** для автоматического определения типа данных:

```
auto array = new int[15][7]; // намного проще!
```

К сожалению, это относительно простое решение не работает, если правый индекс не является константой типа compile-time. В таком случае всё немного усложняется. Сначала мы выделяем массив указателей (как в примере, приведенном выше), а затем перебираем каждый элемент массива указателей и выделяем динамический массив для каждого элемента этого массива. Итого, наш динамический двумерный массив — это динамический одномерный массив динамических одномерных массивов!

```
int **array = new int*[15]; // выделяем массив из 15 указателей типа int - это наши строки
for (int count = 0; count < 15; ++count)
    array[count] = new int[7]; // а это наши столбцы
```

Доступ к элементам массива выполняется как обычно:

```
array[8][3] = 4; // это то же самое, что и (array[8])[3] = 4;
```

Этим методом, поскольку каждый столбец массива динамически выделяется независимо, можно сделать динамически выделенные двумерные массивы, которые не являются прямоугольными. Например, мы можем создать массив треугольной формы:

```
int **array = new int*[15]; // выделяем массив из 15 указателей типа int - это наши строки
for (int count = 0; count < 15; ++count)
    array[count] = new int[count+1]; // а это наши столбцы
```

В примере, приведенном выше, `array[0]` — это массив длиной 1, а `array[1]` — массив длиной 2 и т.д.

Для освобождения памяти динамически выделенного двумерного массива (который создавался с помощью этого способа) также потребуется цикл:

```
for (int count = 0; count < 15; ++count)
    delete[] array[count];
delete[] array; // это следует выполнять в конце
```

Обратите внимание, мы удаляем массив в порядке, противоположном его созданию. Если мы удалим массив перед удалением элементов массива, то нам придется получать доступ к освобожденной памяти для удаления элементов массива. А это, в свою очередь, приведет к неожиданным результатам.

Поскольку процесс выделения и освобождения двумерных массивов является несколько запутанным (можно легко наделать ошибок), то часто проще «сплющить» двумерный массив в одномерный массив:

```
// Вместо следующего:
int **array = new int*[15]; // выделяем массив из 15 указателей типа int - это наши строки
for (int count = 0; count < 15; ++count)
    array[count] = new int[7]; // а это наши столбцы

// Делаем следующее:
int *array = new int[105]; // двумерный массив 15x7 "сплющенный" в одномерный массив
```

Простая математика используется для конвертации индексов строки и столбца прямоугольного двумерного массива в один индекс одномерного массива:

```
int getSingleIndex(int row, int col, int numberOfColumnsInArray)
{
    return (row * numberOfColumnsInArray) + col;
}

// Присваиваем array[9,4] значение 3, используя наш "сплющенный" массив
array[getSingleIndex(9, 4, 5)] = 3;
```

Указатель на указатель на указатель на указатель и т.д.

Также можно объявить указатель на указатель на указатель:

```
int ***ptrx3;
```

Они могут использоваться для динамического выделения трехмерного массива. Тем не менее, для этого потребуется цикл внутри цикла и чрезвычайная аккуратность и осторожность, чтобы не наделать ошибок. Вы даже можете объявить указатель на указатель на указатель на указатель:

```
int ****ptrx4;
```

Или сделать еще большую вложенность, если захотите. Однако на практике такие указатели редко используются.

Заключение

Рекомендуется применять указатели на указатели только в самых крайних случаях, так как они сложны в использовании и потенциально опасны. Достаточно легко разыменовать нулевой или «висячий» указатель в ситуациях с использованием обычных указателей, вдвое легче это сделать в ситуациях с указателем на указатель, поскольку для получения исходного значения потребуется выполнить двойное разыменование!